

CERTIFICAÇÃO LINUX

SELinux

Controle de Acesso Obrigatório (MAC) e RBAC no Linux
— LPIC-3 Security 333.2

Prof. Uirá Ribeiro

Material de apoio do curso Certificação Linux.

Todos os direitos reservados.

Manual de SELinux — Controle de Acesso Obrigatório (MAC) e RBAC no Linux

Adaptado e traduzido a partir do material LPIC-3 Security (303), Tópico 333 — Controle de Acesso, Objetivo 333.2 — Mandatory Access Control (Lessons 1 e 2). Reorganizado em formato de manual de referência, com um resumo de comandos consolidado ao final.

Sumário

1. Introdução: DAC, MAC e RBAC
 2. Mandatory Access Control (MAC) e contextos de segurança
 3. O que é o SELinux
 4. Arquivo de configuração e seleção de política
 5. Inspeccionando o estado operacional
 6. Alterando o modo de imposição em tempo de execução
 7. Estrutura do diretório de configuração
 8. Booleanos e controle condicional de política
 9. Gerenciamento de contextos
 10. Troca de papel (role) e de contexto
 11. Auditoria e análise de política
 12. Alternativas ao SELinux: AppArmor e SMACK
 13. Cheat-sheet de comandos
 14. Exercícios comentados
 15. Resumo geral
-

1. Introdução: DAC, MAC e RBAC

O modelo tradicional do Linux, **Controle de Acesso Discrecionário (DAC — Discretionary Access Control)**, é fácil de entender: dono, grupo e bits de permissão. Porém, ele tem uma fraqueza fundamental: o dono de um recurso pode enfraquecer sua própria proteção. Se um processo é

comprometido (por exemplo, um servidor web explorado por um atacante), esse processo pode acessar tudo o que o usuário dono tem permissão para acessar — nada além do DAC contém esse avanço.

O **Controle de Acesso Obrigatório (MAC — Mandatory Access Control)** existe justamente para impedir esse cenário: as decisões de acesso deixam de ser discricionárias (decididas pelo dono do recurso) e passam a ser centralmente impostas pela política do sistema, mesmo contra a vontade do dono ou de um processo com privilégios de root.

O **Controle de Acesso Baseado em Papéis (RBAC — Role Based Access Control)** complementa esse modelo introduzindo uma camada adicional em que as permissões não são atribuídas diretamente aos usuários, mas a **papéis (roles)**. Os usuários são então associados a papéis, e os papéis determinam quais domínios (tipos) podem ser acessados. Isso limita as ações que um usuário pode executar com base na função atribuída a ele, e não apenas em sua identidade.

Para viabilizar isso, o sistema precisa separar três conceitos:

- um **usuário (user)** é a identidade dentro da política de segurança;
- um **papel (role)** é aquilo que essa identidade tem permissão de assumir;
- e um **tipo (type)** é aquilo que o processo de fato tem permissão de fazer.

Essa estrutura pode parecer complexa, mas ela impõe uma distinção crítica:

DAC responde: *"Você tem permissão?"*

MAC responde: *"Você deveria ter permissão de fazer isso?"*

RBAC responde: *"O que você pode fazer com base no seu papel?"*

A complexidade vem de requisitos de segurança reais:

- isolamento forte entre serviços;
- aplicação do princípio do menor privilégio;
- limitação de danos em caso de comprometimento;
- controle de acesso baseado em responsabilidades definidas (papéis).

Uma forma simples de entender:

- **DAC** = ter a chave de uma porta;
- **MAC** = ter a chave, mas também regras de segurança, câmeras e um guarda;
- **RBAC** = ter um crachá que determina quais áreas você pode entrar, de acordo com sua função.

Você pode até ter acesso, mas isso não significa que pode ir a qualquer lugar — e seu papel restringe ainda mais o que se espera que você faça.

O SELinux impõe isso através de um modelo em camadas:

usuário → papel (role) → tipo (type) → regras da política

Cada camada restringe a seguinte, garantindo que o acesso seja rigidamente controlado e não possa ser arbitrariamente sobreposto.

O **DAC é sempre avaliado primeiro**. Se o DAC nega o acesso, a operação é imediatamente rejeitada. Se o DAC permite o acesso, a política MAC é então avaliada.

Mesmo processos executando como `root` estão sujeitos às políticas MAC e podem ter acesso negado, apesar de terem permissões DAC completas.

O que é Type Enforcement (TE)

O texto original menciona repetidamente que o **tipo (type)** é "o que o processo tem permissão de fazer", mas o mecanismo que efetivamente compara tipos e decide o que é permitido tem nome próprio: **Type Enforcement (TE)**.

TE é o coração técnico do SELinux: toda decisão de acesso é, no fim, uma pergunta sobre dois tipos — "o tipo do processo que está pedindo acesso tem permissão, pela política, de realizar esta operação sobre o tipo do objeto-alvo?". Usuários e papéis (RBAC) restringem **em quais tipos um processo pode operar**; o TE é quem decide, dado um tipo de processo e um tipo de objeto, **se a operação específica é permitida**. É por isso que a maioria das regras de política SELinux (`allow`, `neverallow`, `type_transition`) são, na prática, regras de TE.

2. Mandatory Access Control (MAC) e contextos de segurança

O **MAC** impõe decisões de acesso com base em **rótulos de segurança (security labels)** atribuídos tanto a sujeitos (processos) quanto a objetos (arquivos).

Esses rótulos são interpretados pelo kernel de acordo com uma política de segurança predefinida, que define as interações permitidas entre sujeitos e objetos.

Em sistemas com SELinux, cada objeto tem um contexto de segurança armazenado como um atributo estendido. A opção `-Z` exibe o contexto de segurança do SELinux associado a arquivos ou processos. Esse contexto inclui o usuário SELinux, o papel, o tipo e o nível. É comumente usada com comandos como `ls -Z` e `ps -Z` para inspecionar como as políticas do SELinux estão sendo aplicadas no sistema.

```
$ ls -Z /var/www/html/index.html
system_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html
```

A saída representa o contexto de segurança do SELinux associado ao arquivo. Esse contexto é composto por quatro campos separados por dois-pontos, cada um definindo um aspecto específico do modelo de segurança:

Usuário SELinux (`_u`) O primeiro campo, `system_u`, é o *usuário SELinux*. Ele não corresponde necessariamente a uma conta de usuário Linux, mas define a identidade usada pela política do SELinux. É usado para agrupar papéis e controlar as transições permitidas.

Papel (`_r`) O segundo campo, `object_r`, é o *papel (role)*. No caso de arquivos e outros objetos, normalmente é `object_r`, indicando que a entidade é um recurso, não um processo em execução. Papéis são mais relevantes para sujeitos (processos) do que para objetos.

Tipo (`_t`) O terceiro campo, `httpd_sys_content_t`, é o *tipo (type)*. Este é o campo mais importante do SELinux, pois é usado pelo mecanismo de **Type Enforcement** para determinar as regras de controle de acesso. Neste exemplo, o tipo indica que o arquivo é destinado a ser acessado pelo servidor HTTP.

Nível (level) O quarto campo, `s0`, é o *nível*, também chamado de nível de segurança ou sensibilidade. É usado em sistemas de Segurança Multinível (MLS) ou Multicategoria (MCS) para definir restrições de acesso hierárquicas ou categóricas.

Juntos, esses campos formam o contexto de segurança completo, que o kernel avalia durante as decisões de acesso. A interação entre **o tipo do processo** e **o tipo do arquivo** é o fator principal para determinar se o acesso é permitido pela política do SELinux.

O MAC garante que as decisões de acesso sejam consistentes, centralmente controladas e não possam ser sobrepostas por usuários — incluindo contas privilegiadas como o root.

3. O que é o SELinux

O **SELinux** é um framework de segurança integrado ao kernel que impõe políticas de controle de acesso obrigatório em nível de sistema. Ele opera como parte do kernel Linux, mediando continuamente o acesso entre sujeitos (processos) e objetos (arquivos, sockets e outros recursos), com base em contextos de segurança e regras de política.

O SELinux introduz flexibilidade orientada por política através de **booleanos**, que são flags condicionais que ativam ou desativam regras específicas dentro da política. Esses booleanos permitem ajustar dinamicamente o comportamento do sistema sem exigir a recompilação da política, tornando possível adaptar as restrições de segurança às necessidades operacionais, preservando a integridade do conjunto de regras.

O SELinux normalmente já vem integrado às distribuições Linux modernas, mas sua disponibilidade depende tanto do suporte do kernel quanto das ferramentas de espaço de usuário instaladas. O processo de instalação garante que o sistema tenha os pacotes de política necessários, as ferramentas de gerenciamento e os sistemas de arquivos corretamente rotulados.

Em sistemas onde o SELinux ainda não está instalado, os pacotes necessários incluem a política principal, utilitários de gerenciamento e bibliotecas de suporte. Os nomes exatos dos pacotes dependem da distribuição, mas comumente incluem `selinux-policy`, `selinux-policy-targeted`, `policycoreutils` e `libselinux`.

4. Arquivo de configuração e seleção de política

O arquivo `/etc/selinux/config` define se o SELinux está ativo e qual política é aplicada na inicialização.

```
$ cat /etc/selinux/config
SELINUX=enforcing
SELINUXTYPE=targeted
```

Os estados possíveis para `SELINUX` são `enforcing`, `permissive` e `disabled`. No modo **Enforcing**, as regras de política são aplicadas e as violações de acesso são negadas. No modo **Permissive**, as violações são registradas em log, mas não são impostas. O modo **Disabled** indica que o SELinux não está ativo no kernel.

O `SELINUXTYPE` define a política, como `targeted` ou `mls`.

A **política targeted** é o padrão na maioria das distribuições e aplica restrições do SELinux apenas a serviços e daemons selecionados, como servidores web, servidores de banco de dados e serviços de rede. Processos de usuários comuns e aplicações não confinadas normalmente rodam sem restrições rígidas, equilibrando segurança e usabilidade.

A **política mls** (Multi-Level Security — Segurança Multinível) impõe controle de acesso obrigatório rígido em todos os processos e usuários. Ela introduz níveis de sensibilidade e categorias, garantindo que as decisões de acesso considerem tanto o Type Enforcement quanto os níveis de classificação. Essa política é voltada para ambientes de alta segurança, onde é exigida separação estrita de dados.

5. Inspecionando o estado operacional

O estado operacional atual do SELinux é determinado usando o utilitário `getenforce`:

```
$ getenforce
Enforcing
```

O utilitário `sestatus` fornece uma visão geral completa da configuração e do estado de execução do SELinux:

```
$ sestatus
SELinux status:                enabled
SELinuxfs mount:              /sys/fs/selinux
SELinux root directory:      /etc/selinux
Loaded policy name:          targeted
Current mode:                 enforcing
Mode from config file:       enforcing
Policy MLS status:           enabled
Policy deny_unknown status:   allowed
Memory protection checking:   actual (secure)
Max kernel policy version:    34
```

O utilitário `selinuxenabled` é usado em scripts para determinar se o SELinux está ativo, avaliando seu código de saída em vez de gerar uma saída de texto.

Se o SELinux estiver ativo, o comando retorna status de sucesso (código de saída 0), o que permite que o comando seguinte seja executado ao usar operadores lógicos.

```
$ selinuxenabled && echo "enabled"
enabled
```

6. Alterando o modo de imposição em tempo de execução

O modo de imposição pode ser alterado em tempo de execução usando o utilitário `setenforce`. Essa alteração afeta apenas a sessão atual e não persiste após reinicializações do sistema.

```
$ sudo setenforce 0
$ getenforce
Permissive
```

```
$ sudo setenforce 1
$ getenforce
Enforcing
```

A configuração persistente é controlada através do arquivo `/etc/selinux/config`.

7. Estrutura do diretório de configuração

O diretório `/etc/selinux/` contém os arquivos de configuração principais, os dados de política e os metadados necessários para a inicialização e operação do SELinux. Ele funciona como o local central a partir do qual o sistema determina como o SELinux está configurado e qual política é carregada na inicialização.

Esse diretório inclui o arquivo de configuração principal `/etc/selinux/config`, que define o modo de imposição e o tipo de política selecionado.

Ele também contém subdiretórios específicos de cada política, como `targeted/` e `mls/`, ou `default`, cada um armazenando arquivos de política compilados, definições de contexto e dados auxiliares exigidos pelo motor do SELinux.

Dentro de cada diretório de política, arquivos definem como os rótulos são atribuídos a objetos do sistema de arquivos, como os processos fazem a transição entre domínios e como as decisões de acesso são avaliadas. Isso inclui arquivos de mapeamento de contexto, módulos de política binários e dados de configuração usados durante a inicialização do sistema e operações de re-rotulagem.

Durante a inicialização, o kernel carrega a política selecionada a partir desse diretório, aplica contextos de segurança aos recursos do sistema e inicializa os mecanismos de imposição que irão mediar as decisões de controle de acesso durante toda a execução.

8. Booleanos e controle condicional de política

Os **booleanos do SELinux** são chaves de política configuráveis em tempo de execução que controlam regras condicionais dentro da política.

Eles permitem ativar ou desativar permissões específicas sem modificar ou recompilar a própria política. Esse mecanismo oferece flexibilidade controlada, permitindo que administradores adaptem o comportamento do sistema preservando a integridade e a consistência do modelo de segurança.

O utilitário `getsebool` é usado para consultar o estado dos booleanos, individualmente ou em todo o sistema.

```
$ getsebool httpd_enable_homedirs
httpd_enable_homedirs --> off
```

Essa saída indica que o servidor web (`httpd`) não tem permissão de acessar diretórios home de usuários, de acordo com a configuração atual da política.

Todos os booleanos disponíveis podem ser listados com:

```
$ getsebool -a
allow_ftpd_anon_write --> off
allow_ftpd_full_access --> off
allow_httpd_anon_write --> off
allow_httpd_user_script_anon_write --> off
(...)
```

Esse comando produz uma lista completa de chaves controladas por política, útil para auditoria e para entender as opções de configuração disponíveis.

O utilitário `setsebool` modifica valores de booleanos em tempo de execução. Por padrão, essas alterações afetam apenas a sessão atual e não são preservadas após uma reinicialização.

```
$ sudo setsebool allow_httpd_anon_write on
```

Para tornar a alteração persistente entre reinicializações, usa-se a opção `-P`. Isso grava o valor atualizado no repositório de configuração da política.

```
$ sudo setsebool -P allow_httpd_anon_write on
```

O utilitário `togglesebool` fornece uma interface alternativa que inverte o estado atual de um booleano, sem que seja necessário especificar `on` ou `off` explicitamente. É útil para testar rapidamente o comportamento de uma política.

```
$ sudo togglesebool allow_httpd_anon_write
```

9. Gerenciamento de contextos

Os **contextos do SELinux** são rótulos atribuídos tanto a sujeitos (processos) quanto a objetos (arquivos, diretórios, sockets), que definem como as decisões de controle de acesso são avaliadas. Cada contexto segue a estrutura `usuário:papel:tipo:nível`, e o campo **tipo** é o elemento principal usado nas decisões de Type Enforcement.

O gerenciamento de contexto é feito com utilitários que modificam rótulos diretamente ou os restauram de acordo com as definições da política.

9.1 — Modificando contextos com `chcon`

O utilitário `chcon` modifica diretamente o contexto do SELinux de um arquivo ou diretório, gravando um novo rótulo no atributo estendido associado a esse objeto. É comumente usado para ajustes imediatos, depuração ou validação do comportamento da política.

```
$ chcon -t httpd_sys_content_t /var/www/html/index.html
```

Esse comando altera o campo **tipo** do contexto do arquivo para `httpd_sys_content_t`, o que permite que processos rodando no domínio `httpd_t` acessem o arquivo de acordo com a política do SELinux.

Por padrão, o `chcon` modifica apenas os componentes especificados do contexto, como o tipo, preservando os demais campos (usuário, papel e nível). Opções adicionais permitem especificar o contexto completo, quando necessário.

Alterações feitas com `chcon` são gravadas diretamente no sistema de arquivos e têm efeito imediato. Porém, essas modificações **não são persistentes** do ponto de vista da política. Se uma operação de re-rotulagem for disparada — manualmente, com ferramentas como `restorecon`, ou automaticamente durante a inicialização —, o contexto será redefinido para o valor definido pela política do SELinux.

Atribuições de contexto persistentes devem ser definidas usando ferramentas que conhecem a política, como o `semanage`, que integram as alterações ao sistema de rotulagem do SELinux.

9.2 — Restaurando contextos padrão com `restorecon`

O utilitário `restorecon` restaura o contexto padrão do SELinux com base nos mapeamentos definidos pela política. Ele usa arquivos de configuração de contexto para determinar o rótulo correto de cada arquivo.

```
$ restorecon /var/www/html/index.html
```

Esse comando restaura o contexto do arquivo para o valor esperado pela política, sobrepondo qualquer alteração manual aplicada anteriormente com ferramentas como o `chcon`.

A operação é baseada em caminho e determinística — ou seja, o mesmo caminho de arquivo sempre receberá o mesmo contexto, conforme definido pelos mapeamentos da política.

Para re-rotulagem recursiva de diretórios, a opção `-R` aplica a mesma lógica a todos os arquivos e subdiretórios dentro do caminho especificado:

```
$ restorecon -R /var/www/html/
```

Isso é comumente usado após publicar novo conteúdo, corrigir inconsistências de rotulagem ou aplicar regras de contexto persistentes definidas com `semanage`.

9.3 — Gerenciando contextos persistentes com `semanage`

O utilitário `semanage` gerencia a configuração persistente da política do SELinux, permitindo definir regras personalizadas que sobrevivem a operações de re-rotulagem e a reinicializações do sistema. Ele atua no nível da política, alterando **como os contextos são atribuídos**, em vez de alterar diretamente os rótulos de arquivos individuais.

Diferente do `chcon`, que grava alterações de contexto diretamente nos atributos estendidos do sistema de arquivos, o `semanage` atualiza o banco de dados de mapeamento da política. Isso garante que qualquer operação futura de re-rotulagem — manual ou durante a inicialização do sistema — aplique os contextos definidos de forma consistente.

```
$ sudo semanage fcontext -a -t httpd_sys_content_t "/var/www/html(/.*)?"
$ restorecon -R /var/www/html/
```

Essa sequência cria uma regra persistente que atribui o tipo `httpd_sys_content_t` a todos os arquivos e subdiretórios sob `/var/www/html`. A expressão regular define o escopo da regra, e o `restorecon` aplica os rótulos corretos de acordo com a política atualizada.

O comando `semanage fcontext` modifica a configuração de mapeamento de contexto de arquivos, armazenada na infraestrutura de política do SELinux. Esses mapeamentos são posteriormente usados por ferramentas como `restorecon` e `setfiles` para impor uma rotulagem consistente.

9.4 — Comportamento de contexto com conteúdo web e cópia de arquivos

O Apache HTTPD está instalado e usa a raiz de documentos padrão localizada em `/var/www/html`:

```
$ sudo dnf install httpd
$ sudo systemctl enable --now httpd
```

Ao criar um diretório e um arquivo dentro de `/var/www/html/` com privilégios elevados — quando o diretório está localizado sob a raiz web padrão e corretamente rotulado —, os arquivos recém-criados herdam um contexto de SELinux apropriado:

```
$ sudo mkdir /var/www/html/testdir
$ echo "Hello World" | sudo tee /var/www/html/testdir/index.html
$ ls -Z /var/www/html/testdir/index.html
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/testdir/index.html
```

Como esperado, o servidor web consegue ler o arquivo:

```
$ curl http://localhost/testdir/index.html
Hello World
```

Um resultado diferente aparece quando um arquivo é criado primeiro no diretório home de um usuário e depois copiado para o diretório web. Nesse caso, o arquivo copiado pode manter seu contexto SELinux original, em vez de receber o tipo de conteúdo web esperado.

```
$ echo "Oops, this will not work" > /home/carol/error.html
$ ls -Z /home/carol/error.html
unconfined_u:object_r:user_home_t:s0 /home/carol/error.html
$ sudo cp -a /home/carol/error.html /var/www/html/testdir/error.html
$ ls -Z /var/www/html/testdir/error.html
unconfined_u:object_r:user_home_t:s0 /var/www/html/testdir/error.html
```

Neste exemplo, o contexto original `user_home_t` é preservado durante a cópia porque a opção `-a` ativa o modo arquivo (archive mode), que mantém todos os atributos do arquivo.

Tentar acessar o arquivo copiado a partir do servidor web resulta em uma negação, mesmo que as permissões DAC permitam o acesso.

```
$ curl -I http://localhost/testdir/error.html
HTTP/1.1 403 Forbidden
```

Uma correção pode ser aplicada usando `chcon` para alterar o tipo do arquivo:

```
$ sudo chcon -t httpd_sys_content_t /var/www/html/testdir/error.html
```

Depois dessa alteração, o servidor web consegue acessar o arquivo:

```
$ curl -I http://localhost/testdir/error.html
HTTP/1.1 200 OK
```

Neste caso, a alteração é efetivamente persistente porque alinha o contexto do arquivo com o padrão definido pela política do SELinux para o caminho `/var/www/html/`. Como esse local já está definido na política, não é necessário usar o `semanage` para criar um mapeamento personalizado.

9.5 — Aplicando contextos com `setfiles`

O utilitário `setfiles` aplica contextos do SELinux diretamente a arquivos, com base em um arquivo de mapeamento de política especificado. Ele lê definições de contexto de arquivos como `/etc/selinux/targeted/contexts/files/file_contexts` e atribui rótulos de acordo, sem depender de padrões implícitos do sistema.

```
$ sudo setfiles -v /etc/selinux/targeted/contexts/files/file_contexts /var/www/
```

Esse comando processa todos os arquivos sob `/var/www/` e aplica os contextos apropriados, conforme definidos no arquivo de mapeamento da política. A opção `-v` ativa a saída detalhada, permitindo inspecionar cada operação de rotulagem.

Diferente do `restorecon`, que determina o contexto correto a partir de buscas internas na política, o `setfiles` usa explicitamente o arquivo de contexto fornecido como referência. Isso o torna adequado para operações de re-rotulagem controladas, ambientes personalizados ou cenários de recuperação, em que é necessário controle preciso sobre a rotulagem.

9.6 — Re-rotulagem do sistema de arquivos após habilitar o SELinux

Se o SELinux estava desabilitado anteriormente, habilitá-lo exige uma re-rotulagem completa do sistema de arquivos, para garantir que todos os arquivos tenham contextos de segurança corretos. Isso é feito criando o arquivo `/.autorelabel`, que dispara a re-rotulagem na próxima inicialização.

```
$ sudo touch /.autorelabel
```

Durante a reinicialização, o sistema re-rotula todos os arquivos de acordo com a política selecionada. Essa etapa é crítica, pois rótulos incorretos ou ausentes podem causar negação de acesso ou mau funcionamento do sistema.

O utilitário `fixfiles` é usado para operações de re-rotulagem em grande escala, incluindo a re-rotulagem completa do sistema de arquivos. É comumente usado após habilitar o SELinux ou quando são detectadas grandes inconsistências de rotulagem.

```
$ sudo fixfiles relabel
```

Esse comando agenda ou executa uma re-rotulagem completa do sistema de arquivos, dependendo da configuração do sistema.

10. Troca de papel (role) e de contexto

A **troca de contexto do SELinux** se refere à transição controlada de um processo de um contexto de segurança para outro, dentro dos limites impostos pela política. Cada processo opera sob um contexto composto por usuário, papel, tipo e nível, e qualquer transição entre esses contextos precisa ser explicitamente permitida pela política.

Esse mecanismo é essencial para impor a separação de responsabilidades e o menor privilégio. Em vez de conceder privilégios amplos diretamente, o SELinux permite que processos façam a transição para papéis

ou domínios específicos apenas quando autorizado. Essas transições são governadas pelo **RBAC** e pelo **Type Enforcement (TE)**, garantindo que mudanças de privilégio sejam controladas e auditáveis.

O SELinux fornece diversos utilitários para realizar transições de contexto e de papel, incluindo `newrole`, `runcon` e `setcon`. Cada ferramenta atua em um nível diferente e tem um propósito específico dentro do framework de política.

10.1 — Troca de papel com `newrole`

O utilitário `newrole` é usado para fazer a transição de uma sessão interativa para um papel diferente do SELinux, o que afeta diretamente o domínio no qual os processos têm permissão de executar. Essa transição é rigidamente controlada pela política do SELinux e é um mecanismo central do **RBAC**, no qual os papéis definem quais domínios (tipos) podem ser acessados.

No SELinux, a identidade de um usuário não está diretamente associada a privilégios. Em vez disso, o usuário é associado a um ou mais papéis, e cada papel define um conjunto de transições de domínio permitidas. O papel atual determina quais tipos o processo pode assumir e, portanto, quais ações são permitidas segundo o Type Enforcement.

O contexto atual de uma sessão pode ser inspecionado da seguinte forma:

```
$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Nesse contexto, `unconfined_u` é o usuário SELinux, `unconfined_r` é o papel, e `unconfined_t` é o domínio no qual o shell está sendo executado. O papel `unconfined_r` permite que o processo permaneça em um domínio irrestrito.

Para determinar quais papéis estão disponíveis e são válidos para um determinado usuário SELinux, o utilitário `semanage` é usado para inspecionar os mapeamentos entre usuários Linux e usuários SELinux, bem como os papéis associados a eles.

Para confirmar quais papéis são permitidos para cada usuário SELinux:

```
$ sudo semanage user -l
```

A saída exibe os usuários SELinux disponíveis e os papéis associados a cada um:

SELinux User	SELinux Roles
guest_u	guest_r
root	system_r staff_r sysadm_r unconfined_r
staff_u	system_r staff_r sysadm_r unconfined_r
sysadm_u	sysadm_r
system_u	system_r unconfined_r
unconfined_u	system_r unconfined_r
user_u	user_r
xguest_u	xguest_r

Cada usuário SELinux está vinculado a um conjunto predefinido de papéis, e apenas esses papéis podem ser assumidos durante uma transição. Isso significa que a troca de papel é restrita pela política, não pelos privilégios tradicionais do sistema.

Neste contexto, o usuário SELinux é `unconfined_u`, que está restrito ao papel `unconfined_r`.

Para demonstrar a troca de papel no SELinux, crie um novo usuário `emma`:

```
$ sudo useradd -m emma
$ sudo passwd emma
```

Defina um mapeamento de login explícito do SELinux para esse usuário, associando-o ao usuário SELinux `staff_u`, que permite transições para papéis administrativos:

```
$ sudo semanage login -a -s staff_u emma
```

Esse comando cria um mapeamento entre o usuário Linux `emma` e o usuário SELinux `staff_u`, permitindo que a sessão herde os papéis e as transições de domínio permitidas a `staff_u`, de acordo com a política do SELinux. A opção `-a` adiciona uma nova entrada de mapeamento de login, enquanto a opção `-s` especifica o usuário SELinux a ser associado ao usuário Linux informado.

Verifique o mapeamento:

```
$ sudo semanage login -l
```

Login Name	SELinux User	MLS/MCS Range	Service
__default__	unconfined_u	s0-s0:c0.c1023	*
root	unconfined_u	s0-s0:c0.c1023	*
emma	staff_u	s0-s0:c0.c1023	*

Ao entrar como `emma` (sem usar `su`), confirme o contexto da sessão:

```
$ id -Z
staff_u:staff_r:staff_t:s0-s0:c0.c1023
```

Agora a transição de papel se torna válida:

```
$ newrole -r sysadm_r
$ id -Z
staff_u:sysadm_r:sysadm_t:s0-s0:c0.c1023
```

Esse comando inicia uma transição de papel e normalmente exige autenticação. A política precisa permitir explicitamente que o usuário SELinux entre no papel-alvo, e precisa definir uma transição válida para um domínio associado.

O papel agora é `sysadm_r`, e o processo está sendo executado no domínio `sysadm_t`. Esse domínio está associado a capacidades administrativas, mas o acesso ainda é governado pelas regras da política do SELinux. A transição não concede acesso irrestrito — ela apenas permite ações explicitamente autorizadas para o novo tipo.

Isso demonstra que o RBAC não concede permissões diretamente, mas controla quais domínios um usuário pode acessar. As capacidades efetivas da sessão são determinadas pela combinação de papel e tipo, não apenas pela identidade do usuário.

A troca de papel é comumente usada para separar tarefas administrativas das atividades de usuário comuns, garantindo que privilégios elevados fiquem ativos apenas quando explicitamente solicitados e autorizados pela política.

10.2 — Executando comandos com um contexto específico usando `runcon`

O utilitário `runcon` executa um único comando sob um contexto SELinux especificado. Diferente do `newrole`, que afeta uma sessão interativa inteira, o `runcon` se aplica apenas ao processo invocado.

Isso permite controle preciso sobre como um comando específico é executado, o que é útil para testes, depuração, ou para impor contextos de execução estritos.

```
$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Para executar um comando com um papel SELinux diferente, o utilitário `runcon` pode ser usado. Neste exemplo, o papel é alterado de `unconfined_r` para `system_r`, mantendo o mesmo usuário SELinux, tipo e nível:

```
$ runcon -r system_r id -Z
unconfined_u:system_r:unconfined_t:s0-s0:c0.c1023
```

Para executar um comando com um tipo diferente, o tipo-alvo precisa ser permitido para o papel atual.

10.3 — Modificando o contexto de um processo com `setcon`

O utilitário `setcon` tenta alterar o contexto SELinux do processo atual. Essa operação modifica diretamente o rótulo do processo e está sujeita a restrições rígidas da política.

```
$ setcon system_u:system_r:httpd_t:s0
```

Se permitido, o processo atual adota o novo contexto. No entanto, essa operação é raramente usada na prática, pois exige suporte explícito da política e pode interromper fluxos de execução esperados.

Diferente do `runcon`, que cria um novo processo com um contexto diferente, o `setcon` modifica o contexto de um processo **já existente**. Essa distinção é importante, pois o uso indevido pode gerar comportamento inconsistente ou violações de política.

Por esse motivo, o `setcon` é reservado principalmente para cenários especializados, como testes, depuração ou ambientes controlados. Em sistemas Linux modernos, muitas vezes ele nem vem instalado por padrão, ou é deliberadamente evitado, já que sua capacidade de substituir arbitrariamente todo o contexto SELinux é considerada insegura e incompatível com a aplicação de segurança orientada por política.

10.4 — Restrições de política e transições de contexto

Todas as transições de contexto e de papel no SELinux são governadas por regras de política. Um processo não pode assumir arbitrariamente um novo papel ou tipo; a transição precisa ser explicitamente definida e permitida.

O RBAC determina quais papéis um usuário pode assumir, enquanto o **Type Enforcement (TE)** define quais transições de domínio são permitidas entre tipos. Essas restrições garantem que os caminhos de escalonamento de privilégio sejam rigidamente controlados e que os processos operem apenas dentro dos limites autorizados.

Se uma transição não é permitida, o kernel nega a operação e registra o evento. Esse comportamento reforça o princípio de que o SELinux impõe segurança através de definições explícitas de política, e não de confiança implícita.

11. Auditoria e análise de política

A **auditoria do SELinux** se refere ao processo de coletar, interpretar e analisar eventos de segurança gerados pelo subsistema SELinux. Esses eventos são tipicamente registrados quando o acesso é negado ou quando decisões de política são impostas. O SELinux se integra ao sistema de Auditoria do Linux (Audit), que registra esses eventos em `/var/log/audit/audit.log`.

Quando o SELinux nega uma operação, ele gera uma mensagem **AVC (Access Vector Cache)**, que descreve o contexto de origem, o contexto de destino, a operação tentada e o motivo da negação. Entender essas mensagens é essencial para depuração e para refinar as políticas do SELinux.

Ferramentas de análise de política permitem inspecionar o banco de dados da política do SELinux, possibilitando a verificação de regras, relações e permissões. Juntas, as ferramentas de auditoria e de análise oferecem um fluxo de trabalho completo para diagnosticar e resolver problemas do SELinux.

11.1 — Inspeccionando informações da política com `seinfo`

O utilitário **seinfo** é usado para consultar e exibir informações sobre a política do SELinux carregada. Ele fornece uma visão sobre tipos, papéis, usuários, atributos e regras definidas na política.

```
$ seinfo
Statistics for policy file: /sys/fs/selinux/policy
Policy Version:          33 (MLS enabled)
Target Policy:           selinux
Handle unknown classes:  allow

Classes:          133   Permissions:        454
Sensitivities:     1    Categories:         1024
Types:            5017  Attributes:         253
Users:            8     Roles:              14
Booleans:         343   Cond. Expr.:        375
Allow:            63162 Neverallow:          0
Auditallow:       161   Dontaudit:          8382
Type_trans:       244795 Type_change:         87
Type_member:      35    Range_trans:        5957
Role allow:       37    Role_trans:         418
Constraints:      72    Validatetrans:      0
MLS Constrain:    72    MLS Val. Tran:      0
Permissives:      0     Polcap:             5
Defaults:         7     Typebounds:         0
Allowxperm:       0     Neverallowxperm:    0
Auditallowxperm:  0     Dontauditxperm:     0
Ibendportcon:     0     Ibpkeycon:          0
Initial SIDs:     27    Fs_use:             33
Genfscon:         106   Portcon:            653
Netifcon:         0     Nodecon:            0
```

Esse comando gera um resumo da política, incluindo contagens de tipos, papéis, usuários e regras. Consultas mais específicas podem ser feitas para inspecionar componentes específicos da política.

```
$ seinfo -t
```

Isso exibe todos os tipos definidos na política. Como a imposição do SELinux é baseada principalmente em tipos, essa informação é essencial ao analisar decisões de acesso.

```
$ seinfo -r
```

Isso mostra todos os papéis definidos na política, o que é relevante ao analisar restrições de controle de acesso baseado em papéis.

No SELinux, um papel define quais domínios (tipos) um processo tem permissão de acessar. Essa relação pode ser inspecionada usando a opção `-r` combinada com o nome do papel.

O utilitário **seinfo** também pode ser usado para exibir quais tipos estão associados a um determinado papel:

```
$ seinfo -r staff_r -x
```

A opção `-x` expande a saída e mostra informações detalhadas, incluindo os tipos (domínios) que o papel `staff_r` tem permissão de usar.

A saída inclui uma seção parecida com:

```
Roles: 1
  role staff_r types { staff_t sysadm_t abrt_helper_t (...) };
```

Isso indica que o papel `staff_r` tem permissão de fazer a transição para os tipos listados. Esses tipos definem os domínios nos quais os processos podem ser executados quando operam sob esse papel.

11.2 — Analisando regras de política com `apol`

A ferramenta **apol** faz parte do conjunto SETools e fornece uma interface gráfica para explorar as regras da política do SELinux. Ela permite inspecionar em detalhe regras de permissão (`allow`), transições de domínio e relações entre tipos.

```
$ apol
```

Dentro da interface, é possível consultar a política para determinar se um determinado tipo de origem tem permissão de realizar uma operação sobre um tipo de destino. Isso é particularmente útil para entender por que uma ação é permitida ou negada.

A ferramenta também pode analisar transições de domínio e identificar quais tipos podem transicionar para outros, ajudando a rastrear fluxos de execução dentro do SELinux.

11.3 — Revisando logs de auditoria com `seaudit`

A ferramenta **seaudit** fornece uma interface gráfica para analisar os logs de auditoria do SELinux. Ela lê os dados de auditoria e os apresenta de forma estruturada, facilitando a filtragem e a interpretação de negações AVC.

```
$ seaudit
```

Através dessa interface, mensagens de auditoria podem ser filtradas por tipo de origem, tipo de destino, classe ou resultado. Isso permite identificar com eficiência padrões em operações negadas e isolar interações problemáticas.

O **seaudit** é especialmente útil em ambientes com grandes volumes de dados de auditoria, nos quais a inspeção manual dos logs se torna impraticável.

Nota: a ferramenta **seaudit** foi descontinuada e não é mais mantida ativamente nas cadeias de ferramentas modernas do SELinux. Sua funcionalidade foi substituída por ferramentas mais flexíveis e roteirizáveis dos ecossistemas `auditd` e `SETools`.

Em ambientes atuais, a análise dos logs de auditoria é feita com `ausearch` e `aureport`, enquanto a interpretação de negações e a geração de política ficam a cargo do `audit2why` e do `audit2allow`. Para exploração de política, ferramentas como o `apol` oferecem uma interface mais completa e mantida.

O utilitário **ausearch** é usado para consultar os logs de auditoria em busca de eventos específicos do SELinux, como negações AVC. Por exemplo, para obter mensagens de negação recentes:

```
$ sudo ausearch -m AVC -ts recent
```

```
type=AVC msg=audit(1712345678.123:456): avc: denied { read } for pid=1234 comm="httpd"
name="error.html" dev="xvda1" ino=123456 scontext=system_u:system_r:httpd_t:s0
tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file
```

A mesma consulta pode ser refinada para focar em um processo ou contexto específico:

```
$ sudo ausearch -m AVC -c httpd
```

O utilitário **aureport** fornece relatórios resumidos com base nos logs de auditoria. Por exemplo, para gerar um resumo de negações AVC:

```
$ sudo aureport -a
```

```
AVC Report
=====
# date time event type success
=====
1. 04/06/2026 19:10:23 AVC denied no
2. 04/06/2026 19:11:05 AVC denied no
```

11.4 — Diagnosticando negações com `audit2why`

O utilitário **audit2why** traduz mensagens brutas de log de auditoria em explicações legíveis para humanos. Ele identifica o motivo por trás de uma negação e fornece contexto sobre a regra de política envolvida.

```
$ sudo ausearch -m AVC | audit2why
type=AVC msg=audit(1775396247.058:882): avc: denied { watch } for pid=11525 comm="gmain"
path="/etc/selinux/targeted/policy" dev="sda3" ino=47096 scontext=staff_u:staff_r:staff_t:s0-
s0:c0.c1023 tcontext=system_u:object_r:semanage_store_t:s0 tclass=dir permissive=0
```

Was caused by:

Missing type enforcement (TE) allow rule.

You can use audit2allow to generate a loadable module to allow this access.

Esse comando recupera mensagens AVC recentes e as envia (pipe) para o `audit2why`, que explica por que cada negação ocorreu. A saída geralmente indica se a negação se deve a permissões ausentes, restrições de política ou configurações de booleanos.

Essa ferramenta é útil para entender rapidamente a causa de uma negação, sem precisar interpretar manualmente os logs brutos de auditoria.

11.5 — Gerando regras de política com `audit2allow`

O utilitário **audit2allow** converte mensagens de log de auditoria em regras de política do SELinux. Ele sugere regras `allow` que permitiriam a operação negada.

```
$ sudo ausearch -m AVC | audit2allow
```

A saída mostra regras de política candidatas que poderiam ser adicionadas para permitir a operação. Essas regras precisam ser cuidadosamente revisadas antes do uso, já que aplicá-las às cegas pode enfraquecer o modelo de segurança.

Para gerar um módulo de política carregável:

```
$ sudo ausearch -m AVC | audit2allow -M mymodule
$ sudo semodule -i mymodule.pp
```

Isso cria e instala um módulo de política personalizado. Essa abordagem é útil quando operações legítimas são consistentemente negadas e exigem ajustes explícitos na política.

Atenção: usar `audit2allow` de forma automática, sem investigar a causa raiz da negação, é uma das formas mais comuns de enfraquecer a segurança de um sistema SELinux. Veja a seção de [exercícios comentados](#) para um caso concreto desse problema.

12. Alternativas ao SELinux: AppArmor e SMACK

12.1 — AppArmor

O **AppArmor (Application Armor)** é um Módulo de Segurança do Linux (LSM — Linux Security Module) que implementa controle de acesso obrigatório usando um modelo de segurança baseado em **caminhos de arquivo**. Em vez de rotular objetos do sistema de arquivos como faz o SELinux, o AppArmor associa políticas de segurança a **perfis de aplicação**, que definem quais caminhos de arquivo e recursos uma aplicação pode acessar e como podem ser usados.

Um perfil do AppArmor especifica permissões como leitura, escrita, execução e acesso de rede. Esses perfis são aplicados aos programas no momento da execução, confinando seu comportamento de acordo com regras predefinidas.

Os perfis normalmente ficam armazenados no diretório `/etc/apparmor.d/` e são escritos em um formato legível por humanos, o que facilita seu entendimento e manutenção.

O AppArmor opera em dois modos principais. No **modo enforce (imposição)**, violações de política são bloqueadas e as operações negadas são registradas em log. No **modo complain (reclamação)**, as violações são permitidas, mas registradas, possibilitando testes e ajustes seguros dos perfis sem interromper o funcionamento da aplicação.

O status do AppArmor e de seus perfis pode ser inspecionado com:

```
$ sudo aa-status
```

Os perfis podem ser alternados entre os modos usando:

```
$ sudo aa-complain /etc/apparmor.d/usr.sbin.httpd
$ sudo aa-enforce /etc/apparmor.d/usr.sbin.httpd
```

Esses comandos colocam um perfil em modo complain ou enforce, respectivamente.

O AppArmor também fornece ferramentas para gerar e atualizar perfis com base no comportamento observado da aplicação, permitindo o endurecimento incremental das políticas ao analisar a atividade real e refinar as operações permitidas.

Como o AppArmor depende de caminhos de arquivo, sua eficácia depende da estabilidade e previsibilidade desses caminhos. Mudanças como links simbólicos ou realocação de arquivos podem afetar a imposição da política. Ainda assim, sua simplicidade e facilidade de uso o tornam amplamente adotado em distribuições como o Ubuntu.

O AppArmor oferece uma alternativa prática e acessível ao SELinux, particularmente em ambientes onde se prefere um gerenciamento de política mais simples.

12.2 — SMACK

O **SMACK (Simplified Mandatory Access Control Kernel)** é um Módulo de Segurança do Linux (LSM) que implementa controle de acesso obrigatório usando um modelo simplificado baseado em rótulos. Cada sujeito (como um processo) e cada objeto (como um arquivo ou socket) recebe um rótulo, e as decisões de acesso são tomadas com base em regras explícitas que definem como esses rótulos interagem entre si.

Diferente do SELinux, que usa uma linguagem de política complexa com papéis, tipos e atributos, o SMACK se concentra em um conjunto de regras mínimo e previsível. O núcleo da política SMACK consiste em regras de acesso simples, que especificam qual rótulo pode acessar outro rótulo e quais operações são permitidas, como leitura, escrita ou execução.

Os rótulos do SMACK são armazenados como atributos estendidos nos objetos do sistema de arquivos, tipicamente usando o atributo `security.SMACK64`.

As regras são definidas em um formato simples, no qual são especificados um rótulo de sujeito, um rótulo de objeto e as permissões de acesso. Esse design permite que administradores entendam e modifiquem a política de segurança rapidamente, sem lidar com abstrações complexas.

O SMACK não implementa **RBAC** nem as construções avançadas de política encontradas no SELinux. Em vez disso, ele impõe o acesso estritamente com base em comparações de rótulos e regras predefinidas. Essa simplicidade torna o SMACK adequado para sistemas embarcados, dispositivos móveis e ambientes em que um modelo de segurança leve é necessário.

Devido a seu design direto, o SMACK oferece comportamento previsível e gerenciamento de política mais simples, mas com menos flexibilidade e granularidade em comparação ao SELinux. Seu uso é comum em sistemas especializados, onde simplicidade e desempenho são priorizados em relação a recursos avançados de política.

12.3 — Comparação rápida

	SELINUX	APPARMOR	SMACK
Modelo	Rótulos + Type Enforcement + RBAC	Caminhos de arquivo (perfis por aplicação)	Rótulos simples + regras diretas
Granularidade	Muito alta	Média	Baixa/média

	SELINUX	APPARMOR	SMACK
Curva de aprendizado	Alta	Baixa/média	Baixa
RBAC	Sim	Não	Não
Persistência do rótulo	Atributo estendido, segue o inode	Não aplicável (baseado em caminho)	Atributo estendido (security.SMACK64)
Uso típico	Servidores, distros enterprise (RHEL/Fedora/CentOS)	Desktops/servidores mais simples (Ubuntu/SUSE)	Embarcados, mobile (ex.: Tizen)
Modos de operação	enforcing / permissive / disabled	enforce / complain	Sempre ativo quando carregado

13. Cheat-sheet de comandos

COMANDO	FUNÇÃO
<code>ls -Z, ps -Z, id -Z</code>	Exibe o contexto SELinux de arquivos, processos ou da sessão atual.
<code>getenforce</code>	Mostra o modo de imposição atual (Enforcing/Permissive/Disabled).
<code>setenforce 0 1</code>	Alterna entre Permissive e Enforcing em tempo de execução (não persiste).
<code>sestatus</code>	Mostra visão completa do estado e configuração do SELinux.
<code>selinuxenabled</code>	Retorna código de saída indicando se o SELinux está ativo (uso em scripts).
<code>getsebool [-a] <nome></code>	Consulta o valor de um booleano (ou de todos).
<code>setsebool [-P] <nome> on off</code>	Altera um booleano em tempo de execução (-P = persistente).
<code>togglesebool <nome></code>	Inverte o valor atual de um booleano.
<code>chcon -t <tipo> <arquivo></code>	Altera o contexto de um arquivo diretamente (não persiste a re-rotulagem).
<code>restorecon [-R] <caminho></code>	Restaura o contexto padrão definido pela política.
<code>semanage fcontext -a -t <tipo> "<regex>"</code>	Cria mapeamento persistente de contexto por caminho.

COMANDO	FUNÇÃO
<code>semanage login -a -s <selinux_user> <linux_user></code>	Mapeia um usuário Linux a um usuário SELinux.
<code>semanage user -l / semanage login -l</code>	Lista usuários/papéis SELinux e mapeamentos de login.
<code>setfiles -v <file_contexts> <caminho></code>	Aplica contextos com base em um arquivo de mapeamento explícito.
<code>touch /.autorelabel</code>	Agenda re-rotulagem completa no próximo boot.
<code>fixfiles relabel</code>	Executa/agenda re-rotulagem completa do sistema de arquivos.
<code>newrole -r <papel></code>	Troca o papel SELinux da sessão interativa atual.
<code>runcon -r <papel> <comando></code>	Executa um único comando sob um papel/contexto específico.
<code>setcon <contexto></code>	Altera o contexto do processo atual (uso raro, alto risco).
<code>seinfo, seinfo -t, seinfo -r [-x]</code>	Consulta estatísticas e componentes da política carregada.
<code>apol</code>	Interface gráfica (SETools) para explorar regras de política.
<code>seaudit</code>	Interface gráfica para logs de auditoria (descontinuada).
<code>ausearch -m AVC [-ts recent] [-c <processo>]</code>	Consulta eventos AVC no log de auditoria.
<code>aureport -a</code>	Gera relatório resumido de eventos de auditoria.
<code>audit2why</code>	Traduz mensagens AVC em explicações legíveis.
<code>audit2allow [-M <módulo>]</code>	Gera regras/módulo de política a partir de negações.
<code>semodule -i <módulo>.pp</code>	Instala um módulo de política compilado.
<code>aa-status</code>	Mostra o status do AppArmor e de seus perfis.
<code>aa-complain / aa-enforce <perfil></code>	Alterna um perfil do AppArmor entre modo reclamação/imposição.

14. Exercícios comentados

14.1 — ACL, umask e criação de arquivo (DAC)

Enunciado: um diretório `/projects/dev` está configurado com a seguinte ACL padrão:

```
$ getfacl /projects/dev
# file: projects/dev
# owner: root
# group: developers
# flags: -s-
default:user::rwx
default:group::rwx
default:other:----
```

O sistema tem um `umask` com valor `027`. Um usuário `emma`, do grupo `developers`, cria um novo arquivo `myfile` dentro de `/projects/dev`. Determine as permissões efetivas finais do novo arquivo, considerando a interação entre o modo solicitado, o `umask` e a ACL padrão. Ajuste a configuração para que todos os novos arquivos concedam acesso de escrita ao grupo, independentemente do `umask`.

Resposta: o arquivo é criado como:

```
$ ls -l /projects/dev/myfile
-rw-rw----. 1 emma developers 0 Apr  4 19:09 /projects/dev/myfile
```

O dono é `emma`, e o grupo é `developers`, o que mostra que o bit **SetGID** em `/projects/dev` está funcionando corretamente. Sem o SetGID, o arquivo normalmente herdaria o grupo primário de `emma`, em vez do grupo do diretório.

Os bits de permissão `rw-rw----` mostram que o modo efetivo final inclui acesso de escrita para o grupo. Um arquivo comum é inicialmente solicitado com modo `666`. O `umask 0027` normalmente reduziria isso para `640`. Porém, como o diretório tem uma ACL padrão concedendo permissões ao grupo, a ACL e a máscara resultantes permitem que o bit de escrita do grupo permaneça efetivo, produzindo `660` — a ACL padrão do diretório prevalece sobre o `umask` para os arquivos criados dentro dele.

14.2 — 403 Forbidden causado por contexto SELinux (MAC)

Enunciado: um servidor web roda em um sistema com SELinux habilitado em modo Enforcing. Um arquivo `/var/www/html/app/index.html` existe com permissões DAC corretas, mas o acesso via `curl http://localhost/app/index.html` retorna erro 403 Forbidden. O arquivo foi copiado recentemente de `/home/carol/` usando `cp -a`. Analise a situação inspecionando o contexto SELinux do arquivo e do processo do servidor web em execução. Identifique a causa da negação e aplique uma correção temporária com `chcon`. Em seguida, restaure o contexto correto definido pela política usando `restorecon`, e explique por que a segunda abordagem é preferida em ambientes de produção.

Resposta: a causa é a preservação do contexto original (`user_home_t`) durante a cópia com `cp -a`, já que o modo `archive` mantém os atributos do arquivo de origem — incluindo o contexto SELinux — em vez de aplicar o contexto padrão do diretório de destino.

Correção temporária:

```
$ sudo chcon -t httpd_sys_content_t /var/www/html/app/index.html
```

Correção persistente e alinhada com a política:

```
$ restorecon /var/www/html/app/index.html
$ ls -Z /var/www/html/app/index.html
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/app/index.html
```

O `restorecon` é preferido em produção porque aplica o contexto **definido pela política** para aquele caminho, de forma determinística e reproduzível — o `chcon`, por outro lado, grava um valor manual que será perdido na próxima re-rotulagem do sistema, escondendo o problema em vez de alinhá-lo à política.

14.3 — Mapeamento persistente de contexto para diretório fora do padrão

Enunciado: uma aplicação personalizada armazena conteúdo web em `/srv/webapp/`, que não faz parte da política padrão do SELinux para conteúdo de servidor web. Arquivos colocados nesse diretório não são acessíveis pelo servidor web rodando no domínio `httpd_t`. Defina um mapeamento de contexto SELinux persistente para que todos os arquivos sob `/srv/webapp/` sejam rotulados com o tipo apropriado para conteúdo web. Aplique a configuração usando `semanage`, re-rotule o diretório e verifique se o servidor web consegue acessar o conteúdo.

Resposta:

```
$ sudo semanage fcontext -a -t httpd_sys_content_t "/srv/webapp(/.*)?"
$ sudo restorecon -R /srv/webapp/
$ ls -Z /srv/webapp/index.html
unconfined_u:object_r:httpd_sys_content_t:s0 /srv/webapp/index.html
```

O comando `semanage` cria um mapeamento persistente, garantindo que futuras operações de re-rotulagem mantenham o contexto correto.

14.4 — Booleano para acesso a diretórios home (RBAC/booleanos)

Enunciado: um servidor web roda sob SELinux em modo Enforcing, e surge o requisito de permitir que ele acesse conteúdo armazenado dentro de diretórios home de usuários, como `/home/carol/public_html/`. Por padrão, esse acesso é negado pela política do SELinux. Identifique o booleano SELinux relevante que controla esse comportamento e investigue seu estado atual usando

`getsebool`. Ative o booleano temporariamente e verifique se o servidor web consegue acessar o conteúdo. Em seguida, torne a alteração persistente e observe como o comportamento do SELinux se adapta sem modificar a política principal.

Resposta: o booleano que controla o acesso do servidor web a diretórios home de usuários é

`httpd_enable_homedirs`:

```
$ getsebool httpd_enable_homedirs
httpd_enable_homedirs --> off
```

O booleano pode ser ativado temporariamente:

```
$ sudo setsebool httpd_enable_homedirs on
$ getsebool httpd_enable_homedirs
httpd_enable_homedirs --> on
```

Para garantir que a alteração persista entre reinicializações, usa-se a opção `-P`:

```
$ sudo setsebool -P httpd_enable_homedirs on
```

14.5 — Papéis permitidos para `staff_r` (RBAC)

Enunciado: um administrador do sistema precisa verificar se o papel SELinux `staff_r` tem permissão de fazer a transição para domínios administrativos, como `sysadm_t`. O objetivo é inspecionar a política do SELinux e identificar quais tipos estão associados ao papel `staff_r`, confirmando se essa transição é permitida.

Resposta:

```
$ seinfo -r staff_r -x
```

A saída lista os tipos associados a `staff_r`. Se `sysadm_t` aparecer na lista, o papel tem permissão de fazer a transição para esse domínio.

14.6 — Transição de sessão para papel administrativo

Enunciado: uma sessão de usuário está rodando sob o contexto `staff_u:staff_r:staff_t:s0-s0:c0.c1023`. O objetivo é fazer a transição da sessão para um papel administrativo, a fim de executar operações privilegiadas, assumindo que a política do SELinux permite essa transição.

Resposta:

```
$ newrole -r sysadm_r  
$ id -Z  
staff_u:sysadm_r:sysadm_t:s0-s0:c0.c1023
```

A sessão transiciona para o papel `sysadm_r` e para o domínio associado `sysadm_t`, refletindo as capacidades administrativas elevadas definidas pela política.

14.7 — Diagnóstico de negação AVC

Enunciado: ocorre uma negação do SELinux quando um servidor web tenta acessar um arquivo, e o administrador precisa determinar a causa da negação. O objetivo é recuperar mensagens AVC recentes e interpretá-las em formato legível.

Resposta:

```
$ sudo ausearch -m AVC -ts recent | audit2why
```

Esse comando recupera negações AVC recentes e explica o motivo de cada negação, indicando se ela é causada por regras ausentes, restrições de política ou outros fatores.

14.8 — Geração de módulo de política a partir de negações

Enunciado: negações repetidas do SELinux são observadas para uma aplicação legítima, e o administrador decide gerar um módulo de política personalizado para permitir o acesso necessário. A tarefa é converter os logs de auditoria em um módulo de política SELinux carregável.

Resposta:

```
$ sudo ausearch -m AVC -ts recent | audit2allow -M mymodule  
$ sudo semodule -i mymodule.pp
```

Isso cria e instala um módulo de política que permite as operações anteriormente negadas, com base nas mensagens de auditoria observadas.

14.9 — Execução de comando com papel temporário

Enunciado: um comando precisa ser executado sob um papel SELinux específico, sem afetar a sessão interativa atual. O objetivo é rodar um comando com um papel diferente usando uma transição de contexto temporária.

Resposta:

```
$ runcon -r system_r id -Z
unconfined_u:system_r:unconfined_t:s0-s0:c0.c1023
```

O comando roda com o papel especificado `system_r`, demonstrando uma alteração temporária de contexto que se aplica apenas ao processo executado.

14.10 — Avaliação crítica do uso de `audit2allow` (exercício exploratório)

Enunciado: um administrador observa negações AVC repetidas do SELinux, envolvendo um processo rodando no domínio `httpd_t` tentando acessar arquivos rotulados com `user_home_t`. O administrador considera usar `audit2allow` para gerar um módulo de política que permita esse acesso. Avalie essa abordagem sob a perspectiva de segurança e design de política, explicando se essa é uma solução apropriada e quais ações alternativas deveriam ser consideradas.

Resposta: o uso do `audit2allow` nesse cenário precisa ser avaliado com cuidado, pois ele gera regras `allow` baseadas exclusivamente nas negações observadas, sem considerar o modelo de segurança mais amplo. Permitir que `httpd_t` acesse objetos rotulados como `user_home_t` enfraqueceria os limites de isolamento, já que o tipo `user_home_t` é destinado a conteúdo controlado pelo usuário, e não foi projetado para ser acessado por processos de servidor web.

Uma abordagem mais apropriada é analisar a causa raiz da negação. Neste caso, o problema normalmente surge porque arquivos destinados a ser servidos pelo servidor web estão rotulados incorretamente. Em vez de modificar a política, a solução correta é garantir que os arquivos sob `/var/www/html` estejam rotulados com o tipo apropriado, como `httpd_sys_content_t`.

Isso pode ser feito restaurando o contexto correto com:

```
$ restorecon -Rv /var/www/html
```

Isso garante que a rotulagem esteja alinhada com a política do SELinux, preservando os limites de segurança e, ao mesmo tempo, permitindo o acesso pretendido.

Regra geral: `audit2allow` diz **o que faria a negação desaparecer**, não **o que é seguro permitir**. Trate toda regra sugerida por ele como uma hipótese a ser validada contra o modelo de segurança — nunca como uma correção pronta para aplicar.

15. Resumo geral

- **DAC** decide com base em posse (ownership) e bits de permissão; pode ser enfraquecido pelo próprio dono do recurso ou por um processo comprometido.

- **MAC** impõe decisões de acesso centralmente definidas, usando rótulos de segurança, e não pode ser sobreposto por usuários — nem mesmo pelo root.
- **RBAC** restringe ações mapeando usuários a papéis e controlando quais domínios (tipos) cada papel pode acessar.
- **Type Enforcement (TE)** é o mecanismo que, de fato, decide se um tipo de processo pode operar sobre um tipo de objeto — é o núcleo técnico por trás da maioria das regras SELinux.
- O contexto de segurança do SELinux é composto por `usuário:papel:tipo:nível`, inspecionável com `ls -Z`, `ps -Z` e `id -Z`.
- DAC, RBAC e MAC são avaliados em um modelo em camadas: **todos** precisam permitir o acesso para que uma operação seja bem-sucedida — DAC primeiro, depois MAC.
- O estado do SELinux é inspecionado com `getenforce`, `sestatus` e `selinuxenabled`, e o modo de imposição é controlado em runtime com `setenforce` (persistente em `/etc/selinux/config`).
- **Booleanos** ativam/desativam regras condicionais sem recompilar a política (`getsebool`, `setsebool -P`, `togglesebool`).
- Contextos são gerenciados com `chcon` (imediato, não persistente), `restorecon` (restaura o padrão da política), `semanage` (define mapeamentos persistentes), `setfiles` (aplica a partir de um arquivo de contexto explícito) e `fixfiles / /.autorelabel` (re-rotulagem em massa).
- Transições de papel e contexto são feitas com `newrole` (sessão interativa), `runcon` (processo único) e `setcon` (raramente usado, alto risco) — todas governadas por RBAC e TE.
- Negações geram mensagens **AVC**, analisadas com `ausearch` / `aureport` (consulta/relatório), `audit2why` (explicação) e `audit2allow` (sugestão de regra) — sempre com revisão crítica antes de aplicar.
- **AppArmor** (baseado em caminho, mais simples) e **SMACK** (baseado em rótulos simples, sem RBAC) são LSMs alternativos ao SELinux, adequados a contextos onde simplicidade é priorizada sobre granularidade.

Referências

- LPIC-3 Security (303), Topic 333 — Access Control, Objective 333.2 — Mandatory Access Control (material-fonte deste manual).
- `man selinux`, `man semanage`, `man restorecon`, `man audit2allow`.
- Documentação oficial do projeto SELinux e do SETools.
- Documentação oficial do AppArmor (`man apparmor`) e do projeto SMACK no kernel Linux.